

Unit I: Introduction to Information Security (15 Hours)

1. Introduction to Information Security

- **The Need for Security:** As information is increasingly stored and transmitted digitally, protecting it from unauthorized access, tampering, or theft is critical. Security ensures trust in systems like online banking, e-commerce, and communication platforms.

1. The Need for Security:

- **Importance:**
 - Protects sensitive data: Personal (e.g., Social Security numbers), financial (e.g., credit card details), organizational (e.g., trade secrets).
 - Prevents financial loss: Cybercrime costs trillions annually (e.g., \$6 trillion globally in 2021, per Cybersecurity Ventures).
 - Ensures compliance: Laws like GDPR, HIPAA mandate data protection.
- **Threat Landscape:**
 - Cyberattacks: Hacking, phishing, ransomware.
 - Physical threats: Theft of devices, natural disasters.
 - Human errors: Misconfigured servers, weak passwords.

2. Principles of Security (CIA Triad):

- **Confidentiality:**
 - Ensures data is only accessible to authorized users.
 - Mechanism: Encryption (e.g., AES for encrypting files).
 - Example: A hospital encrypts patient records to prevent unauthorized access.
- **Integrity:**
 - Ensures data is accurate and unaltered.
 - Mechanism: Hash functions (e.g., SHA-256 to verify file integrity).
 - Example: A bank uses checksums to ensure transaction data isn't tampered with.
- **Availability:**
 - Ensures data/systems are accessible when needed.
 - Mechanism: Redundancy, DDoS protection.
 - Example: A website uses a content delivery network (CDN) to stay online during traffic spikes.

- **Additional Principles:**
 - **Authenticity:** Verifies the identity of users/systems (e.g., digital certificates).
 - **Non-repudiation:** Ensures parties cannot deny actions (e.g., digital signatures).

3. Types of Attacks:

- **Passive Attacks:**
 - Goal: Gather information without altering data.
 - Example: Eavesdropping on unencrypted Wi-Fi to steal passwords.
 - Defense: Use encryption (e.g., WPA3 for Wi-Fi).
- **Active Attacks:**
 - Goal: Alter or disrupt data/systems.
 - Example: Man-in-the-middle (MITM) attack where an attacker intercepts and modifies messages.
 - Defense: Use secure protocols (e.g., HTTPS).
- **Virus:**
 - Malicious code that spreads by attaching to programs.
 - Example: The ILOVEYOU virus (2000) spread via email, deleting files.
 - Defense: Antivirus software, user education.
- **Worm:**
 - Self-replicating malware that spreads independently.
 - Example: WannaCry (2017) exploited Windows vulnerabilities, encrypting data for ransom.
 - Defense: Patch management, firewalls.
- **Trojan Horse:**
 - Malware disguised as legitimate software.
 - Example: A fake banking app that steals credentials.
 - Defense: Download software from trusted sources.

4. Introduction to Cryptography:

- **Definition:** The science of securing communication by converting plaintext into ciphertext.
- **Goals:**
 - Confidentiality, integrity, authentication, non-repudiation.

- **Historical Context:**
 - Ancient cryptography: Caesar cipher used by Julius Caesar (shift letters by a fixed number).
 - World War II: Enigma machine used by Germany, broken by Alan Turing.
- **Basic Terms:**
 - Plaintext: Original message (e.g., "HELLO").
 - Ciphertext: Encrypted message (e.g., "KHOOR" with a Caesar shift of 3).
 - Key: Secret used to encrypt/decrypt (e.g., shift value in Caesar cipher).
- **Types of Cryptography:**
 - Symmetric: Same key (e.g., AES).
 - Asymmetric: Public/private keys (e.g., RSA).
 - Hash Functions: No key (e.g., SHA-256).

5. Steganography:

- **Definition:** Hiding a message within another medium (e.g., an image, audio file).
- **Example:**
 - Hide a message in an image by altering the least significant bits (LSBs) of pixel values.
 - Original pixel: 10110110 (182); Modified pixel: 10110111 (183) to hide a bit.
- **Comparison with Cryptography:**
 - Cryptography: Scrambles the message (visible but unreadable).
 - Steganography: Hides the message's existence (invisible).
- **Applications:**
 - Covert communication: Spies hiding messages in images.
 - Digital watermarking: Embedding copyright info in media.

Diagram (Text Description):

- **Cryptography vs. Steganography:**
 - Cryptography: Plaintext → [Encryption with Key] → Ciphertext → [Decryption with Key] → Plaintext.

- Steganography: Message → [Embed in Image] → Stego-Image → [Extract from Image] → Message.
-

Unit II: Symmetric Key Encipherment (20 Hours)

Overview: This unit explores symmetric key cryptography, traditional ciphers, cryptanalysis, and modern stream/block ciphers.

1. Traditional Symmetric Key Ciphers:

- **Introduction:**
 - Symmetric key cryptography uses one key for both encryption and decryption.
 - Historically significant: Used in ancient times (e.g., Scytale by Spartans, a transposition cipher).
- **Kirchhoff's Principle:**
 - The security of a cryptosystem should rely on the secrecy of the key, not the algorithm.
 - Example: AES is a public algorithm, but its security depends on the key.
- **Advantages:**
 - Fast: Suitable for encrypting large data (e.g., AES in disk encryption).
- **Disadvantages:**
 - Key distribution: How to securely share the key?
 - Scalability: n users need $\frac{n(n-1)}{2}$ keys for pairwise communication.

2. Cryptanalysis:

- **Definition:** The study of breaking ciphers without the key.
- **Types of Attacks:**
 - **Ciphertext-Only Attack:** Attacker has only ciphertext.
 - Example: Frequency analysis on a Caesar cipher (e.g., 'E' is the most frequent letter in English).

- **Known-Plaintext Attack:** Attacker has plaintext-ciphertext pairs.
 - Example: Knowing “HELLO” encrypts to “KHOOR” reveals a Caesar shift of 3.
- **Chosen-Plaintext Attack:** Attacker can choose plaintexts to encrypt.
 - Example: Used to break early versions of WEP (Wi-Fi encryption).
- **Goal:** Recover the key or plaintext.

3. Categories of Traditional Ciphers:

- **Substitution Ciphers:**
 - Replace each letter with another letter or symbol.
 - **Mono-alphabetic Ciphers:**
 - Single substitution rule.
 - Example: Caesar cipher (shift by 3: $A \rightarrow D, B \rightarrow E$).
 - **Encryption:** $C = (P + k) \bmod 26$ where P is the plaintext letter ($A=0, Z=25$), k is the shift.
 - **Decryption:** $P = (C - k) \bmod 26$
 - Weakness: Frequency analysis (e.g., ‘E’ appears 12.7% in English text).
 - **Poly-alphabetic Ciphers:**
 - Multiple substitution rules.
 - Example: Vigenère cipher.
 - **Key:** A keyword (e.g., “KEY”).
 - **Encryption:** Shift each letter by the corresponding key letter ($K=10, E=4, Y=24$).
 - Plaintext: “HELLO” → Key: “KEYKE” → Shifts: 10, 4, 24, 10, 4 → Ciphertext: “RILOI”.
 - **Decryption:** Reverse the shifts.
 - Strength: Reduces frequency analysis effectiveness.
 - Weakness: Can be broken with Kasiski examination (finds key length by repeating patterns).
- **Transposition Ciphers:**
 - Rearrange the letters of the plaintext.
 - **Key-less Transposition:**
 - Example: Rail Fence cipher.

Plaintext: "HELLO WORLD" → Write in zigzag (depth 2):

```
text
CollapseWrapCopy
H . L . W . R . D . E . L . O . L .
```

Ciphertext: "HLWRD ELOL".

- **Keyed Transposition:**
 - Example: Columnar transposition.
 - Key: "ZEBRA" (Z=5, E=2, B=1, R=4, A=3 → Order: 2, 1, 3, 4, 5).

Plaintext: "DEFENDTHEEAST" → Write in rows (5 columns):

```
text
CollapseWrapCopy
D E F E N D T H E E A S T
```

Read by key order: "EDDAT FHTSE NEE".

- **Comparison:**
 - Substitution: Changes letters (e.g., A → D).
 - Transposition: Rearranges letters (e.g., HELLO → EHOLL).

4. Stream and Block Ciphers:

- **Stream Ciphers:**
 - Encrypt one bit/byte at a time.
 - Example: RC4 (used in early SSL/TLS).
 - **Mechanism:** Generate a keystream (pseudo-random bits) and XOR with plaintext.
 - **Encryption:** $C = P \oplus K$ $C = P \oplus K$, where P is plaintext, K is keystream.
 - **Decryption:** $P = C \oplus K$ $P = C \oplus K$.
 - Advantage: Fast for real-time applications (e.g., streaming).
 - Weakness: Keystream reuse leads to vulnerabilities (e.g., $C_1 \oplus C_2 = P_1 \oplus P_2$).
- **Block Ciphers:**
 - Encrypt fixed-size blocks (e.g., 64-bit for DES, 128-bit for AES).
 - Example: AES (Advanced Encryption Standard).
 - **Modes of Operation:**

- **ECB (Electronic Codebook):** Each block encrypted independently (weak, patterns visible).
 - **CBC (Cipher Block Chaining):** XORs previous ciphertext with current plaintext (more secure).
 - **CTR (Counter):** Turns block cipher into a stream cipher (parallelizable).
- Advantage: Stronger security for large data.
 - Weakness: Slower than stream ciphers.

Diagram (Text Description):

- **Stream Cipher:**
 - Plaintext → [XOR with Keystream] → Ciphertext.
 - Keystream generated by a pseudo-random generator using the key.
- **Block Cipher (CBC Mode):**
 - Plaintext Block 1 → [XOR with IV] → [Encrypt with Key] → Ciphertext Block 1
 - Ciphertext Block 1 → [XOR with Plaintext Block 2] → [Encrypt with Key] → Ciphertext Block 2.

Data Encryption Standard (DES)

Introduction

The Data Encryption Standard (DES), standardized in 1977 by the National Institute of Standards and Technology (NIST), is a symmetric-key block cipher that encrypts 64-bit plaintext blocks using a 56-bit key (64 bits total, with 8 parity bits). Initially designed for hardware efficiency, DES became a cornerstone of early cryptography but is now obsolete due to its susceptibility to brute-force attacks with modern computing power.

DES Structure

- **Initial and Final Permutations:** The 64-bit plaintext undergoes an Initial Permutation (IP) based on a fixed table (e.g., bit 58 moves to position 1, bit 50 to 2). This does not enhance security but optimizes hardware implementation. After 16 rounds, the Final Permutation (FP), the inverse of IP, restores the bit order.
- **DES Function:** Each of the 16 rounds processes the 64-bit block into two 32-bit halves (L and R). For each round:
 1. **Expansion:** The right half (R) expands from 32 to 48 bits using an expansion table (e.g., bit 32 repeats as bit 1 and 2).

2. **Key Mixing:** XORed with a 48-bit round subkey.
3. **Substitution:** Passes through 8 S-boxes (6-bit input to 4-bit output), providing non-linearity (e.g., S1: 14, 4, 13, 1, ...).
4. **Permutation:** A 32-bit permutation shuffles the output.
5. The halves swap after each round (except the last), with $L_{i+1} = R_i$, $R_{i+1} = L_i \oplus f(R_i, K_i)$

Round Key Generation

- From the 64-bit input key, 8 parity bits are discarded, leaving 56 bits.
- Permuted via PC-1 (e.g., bit 57 to 1), split into two 28-bit halves (C and D).
- Each half is left-shifted (1 or 2 bits, per round schedule: 1, 1, 2, 2, ...).
- Subkeys (48 bits) are selected via PC-2 permutation (e.g., bit 14 to 1).
- Total: 16 unique subkeys, each 48 bits.

Avalanche and Completeness Effect

- **Avalanche Effect:** A 1-bit plaintext change (e.g., flipping bit 1) alters ~50% of ciphertext bits after a few rounds, due to diffusion from expansion and S-boxes.
- **Completeness Effect:** Each ciphertext bit depends on all plaintext and key bits, achieved through iterative permutations and substitutions.

weak Keys

- **Weak Keys:** Four exist (e.g., all 0s: 0x0000000000000000, all 1s: 0xFFFFFFFFFFFFFFFF), producing identical subkeys across rounds, enabling trivial decryption.
- **Semi-Weak Keys:** Six pairs (e.g., 0x01FE01FE01FE01FE) yield only two distinct subkeys, weakening encryption.
- Avoided by key validation in implementations.

multiple DES

- **Double DES:** Encrypt with K1, then K2 ($C = E_{K2}(E_{K1}(P))$). Key space: 2^{112} , but meet-in-the-middle attack reduces effective strength to 2^{57} by storing intermediate results.
- **Triple DES (3DES):** Three modes:
 1. $E_{K1}(D_{K2}(E_{K1}(P)))$ (two keys, 112 bits effective).
 2. $E_{K1}(D_{K2}(E_{K3}(P)))$ (three keys, 168 bits).
 - Slower but secure; used in banking (e.g., EMV cards).

Security of DES

- **Brute-Force Attack:** 2^{56} keys (~72 quadrillion). In 1998, EFF's "Deep Crack" cracked DES in 56 hours; today, it's seconds with specialized hardware.
 - **Differential Cryptanalysis:** Biham and Shamir (1990) exploit plaintext-ciphertext pair differences. Requires 2^{47} chosen plaintexts; DES resists due to S-box optimization (e.g., S5 resists linear biases).
 - **Linear Cryptanalysis:** Matsui (1993) uses linear approximations (e.g., $P[4] \oplus C[15] \approx K[22]$). Needs 2^{43} known plaintexts; mitigated by S-box non-linearity.
-

Public Key Cryptosystem

Principles

Symmetric systems (like DES) use one key; public key systems use:

- **Public Key:** Shared (e.g., for encryption).
- **Private Key:** Secret (e.g., for decryption). Based on trapdoor functions (easy to compute, hard to reverse without the private key).

Applications

- **Encryption:** SSL/TLS secures web traffic (e.g., RSA encrypts AES keys).
- **Digital Signatures:** RSA signs hashes (e.g., SHA-256) for software verification.
- **Key Exchange:** Diffie-Hellman or RSA establishes shared secrets.

Requirements

- **Key Pair Generation:** (public,private) (public, private) (public,private) must be unique and related (e.g., RSA: $e \cdot d \equiv 1 \pmod{\phi(n)}$).
- **Security:** Private key infeasible to derive (e.g., factoring $n=2048$ bits takes 10^{20} years classically).
- **Efficiency:** Fast encryption/decryption (e.g., RSA uses small $e=65537$).

Public Key Cryptanalysis

- **Factoring:** RSA's $n=p \times q$ (e.g., 2048-bit n resists GNFS).
- **Discrete Log:** Attacks Diffie-Hellman (e.g., index calculus).
- **Quantum Threat:** Shor's algorithm factors n in polynomial time (e.g., 2048-bit RSA broken with ~4096 qubits).

RSA Algorithm

Description

RSA leverages prime factorization difficulty:

- **Key Generation:**
 1. Pick primes $p=61$ $p = 61$ $p=61$, $q=53$ $q = 53$ $q=53$.
 2. $n=61 \times 53 = 3233$ $n = 61 \times 53 = 3233$ $n=61 \times 53 = 3233$,
 $\phi(n)=(60)(52)=3120$ $\phi(n) = (60)(52) = 3120$ $\phi(n)=(60)(52)=3120$.
 3. $e=17$ $e = 17$ $e=17$ (coprime with 3120).
 4. $d=2753$ $d = 2753$ $d=2753$ (since $17 \times 2753 \equiv 1 \pmod{3120}$ $17 \times 2753 \equiv 1 \pmod{3120}$).
 - Public: (17,3233) (17, 3233) (17,3233), Private: (2753,3233) (2753, 3233) (2753,3233).
- **Encryption:** $M=65$ $M = 65$ $M=65$, $C=65^{17} \pmod{3233} = 2790$ $C = 65^{17} \pmod{3233} = 2790$ $C=65^{17} \pmod{3233} = 2790$.
- **Decryption:** $M=2790^{2753} \pmod{3233} = 65$ $M = 2790^{2753} \pmod{3233} = 65$ $M=2790^{2753} \pmod{3233} = 65$.

Security of RSA

- **Strength:** Factoring n n n (e.g., 2048 bits) requires 2^{100} operations.
- **Weaknesses:**
 - Small e e e (e.g., 3) risks attacks if $M^e < n$ $M^e < n$ $M^e < n$.
 - Padding (e.g., OAEP) prevents chosen-ciphertext attacks.
- **Future:** Quantum computers (e.g., 10,000 qubits) could break RSA by 2030s.

Digital Signatures

1. Comparison Between Conventional and Digital Signatures

Conventional Signatures:

Nature: Physical (e.g., ink on paper).

Verification: Relies on visual inspection or expert analysis (e.g., handwriting comparison).

Security: Prone to forgery (e.g., imitation of signature style, unauthorized use).

Binding: Legally ties signer to document but lacks inherent proof of authenticity.

Example: Signing a contract with a pen.

Digital Signatures:

Nature: Cryptographic (e.g., hash encrypted with private key).

Verification: Mathematically proven via public key (e.g., RSA decryption).

Security: Resistant to forgery due to cryptographic strength (e.g., factoring large numbers is infeasible).

Binding: Provides **non-repudiation** via signer's unique private key.

Example: Signing an email with a digital certificate (e.g., S/MIME, SSL/TLS authentication).

Key Differences:

Feature	Conventional Signatures	Digital Signatures
Medium	Physical	Electronic
Tamper Evidence	No inherent detection	Detects alterations via cryptographic hash
Scalability	Manual verification	Automated via software
Duplication	Can be copied/forged	Unique per document due to hash dependency

2. Inclusion, Verification, Relationship, and Duplicity

Inclusion: Digital signatures are embedded in or attached to the document (e.g., PDF signature).

Verification: Uses the signer's **public key** to verify authenticity (e.g., $H(M) = D_{pub}(S)$).

Relationship: Cryptographically links signer to document (e.g., private key proves authorship).

Duplicity: Digital signatures are unique per document, preventing replication, unlike conventional signatures, which can be photocopied or traced.

3. Process: Needs for Keys, Signing the Digest

Needs for Keys:

Private Key: Held by the signer, used to create the signature (e.g., encrypt hash)

Public Key: Shared with verifiers, used to validate the signature (e.g., decrypt hash).

Asymmetric Cryptography: Ensures only the signer can sign, but anyone can verify

Signing the Digest:

Hashing: Compute a fixed-length digest using SHA-256:

Example: $M = \text{"Hello"}, H(M) = 0xA591A6D40\dots$

Signing: Encrypt digest with private key using RSA:

Example: $S = H(M)^d \pmod{n}$

Transmission: Send message M and signature S together.

Why Digest?: Hashing reduces computational overhead (e.g., 256-bit vs. full message).

1.

4. Services Provided by Digital Signatures

Service	Purpose	Example
Message Authentication	Verifies signer's identity	Email from Alice signed with her private key
Message Integrity	Ensures document wasn't altered	Changing Pay \$100 to \$1000 alters hash, flagging tampering
Non-Repudiation	Prevents signer from denying signature	Alice's private key uniquely signs the document
Confidentiality	Ensures privacy with encryption	Signed and encrypted emails

5. Digital Signature Forgery and Types

Existential Forgery: Attacker generates a valid (message, signature) pair without controlling message content.

Selective Forgery: Attacker forges a signature for a specific message.

Universal Forgery: Attacker derives private key or equivalent, allowing signing of any message.

Mitigation: Strong cryptography (e.g., RSA-2048, SHA-256, PSS padding).

6. Digital Signature Scheme: RSA Example

Key Generation:

Choose primes: $p = 61, q = 53$

Compute $n = p \times q = 3233, \phi(n) = (p-1)(q-1) = 3120$

Select $e = 17$ ($\gcd(17, 3120) = 1$)

Compute $d = 2753$ (modular inverse of $e \bmod \phi(n)$)

Public key: $(e, n) = (17, 3233)$, **Private key:** $(d, n) = (2753, 3233)$

Signing:

Compute hash $H(M) = 65$ (simplified for example)

Compute $S = H(M)^d \bmod n = 65^{2753} \bmod 3233 = 2790$

Verification:

Compute $H(M)$

Decrypt signature: $S^e \bmod n = 2790^{17} \bmod 3233 = 65$

If $H(M) = S^e \bmod n$, signature is valid.

Security Considerations:

Strength: Relies on factoring n (e.g., RSA-2048 requires 2^{100} operations).

Weaknesses:

No Padding: Raw $H(M)$ signatures risk existential forgery.

Small e : $e=3, H(M)^3 < n$ allows cube root attack.

Solution: Use Probabilistic Signature Scheme (PSS) with random padding.

Quantum Threat: Shor's algorithm (4096 qubits) can break RSA.

7. Critical Insights for Extra Marks

Historical Context:

Digital signatures emerged with public-key cryptography (Diffie-Hellman, 1976; RSA, 1978).

Enabled **secure communications** (e.g., SSL/TLS, PGP, blockchain smart contracts).

Comparison with DES:

Feature	DES (Symmetric)	RSA Digital Signatures (Asymmetric)
Encryption Type	Shared key (same for encryption/decryption)	Key pair (public/private)
Security	No non-repudiation	Provides non-repudiation
Use Case	Confidentiality	Authentication and integrity

Modern Relevance:

RSA-2048 is standard in PKI (e.g., **X.509 certificates** for websites, e-signatures in banking).

Post-quantum cryptography: Algorithms like **Dilithium** are emerging due to quantum threats.

Practical Example

Software Signing: Microsoft signs Windows updates using **RSA-SHA256** to ensure they are genuine and untampered

Blockchain: Bitcoin transactions use **ECDSA digital signatures** for verifying ownership.